MINISTRY OF EDUCATION & TRAINING

HO CHI MINH CITY UNIVERSITY OF ECONOMICS AND FINANCE

FINAL PROJECT

Course: 2025-2026

Course code: 252.ITE1231E

SOFTWARE QUALITY VERIFICATION

Student's Name:	**Võ Gia Huy**	**225050394**
	Trần Nguyễn Diệu Thuý	**225054118**
	Nguyễn Hoàng Phúc	**225052366**

Class: 252.ITE1231E.A04E

Ho Chi Minh City - 4, 2026

MINISTRY OF EDUCATION & TRAINING

HO CHI MINH CITY UNIVERSITY OF ECONOMICS AND FINANCE

FINAL PROJECT

Course: 2025-2026

Course code: 251.ITE1231E

SOFTWARE QUALITY VERIFICATION

Student's Name:	**Võ Gia Huy**	**225050394**
	Trần Nguyễn Diệu Thuý	**225054118**
	Nguyễn Hoàng Phúc	**225052366**

Instructor: Nguyen Phuoc Vinh Loc

Ho Chi Minh City - 4, 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our deepest gratitude to our Instructor, whose dedicated guidance, insightful advice, and profound expertise were instrumental throughout the duration of this testing project. Your practical feedback and strategic direction helped us not only complete this report successfully but also gain a deeper understanding of software quality assurance in a real-world context.

We would also like to extend our thanks to the Board of Management of the Ho Chi Minh City University of Economics and Finance (UEF) and the faculty members of the Information Technology Department for providing a supportive learning environment, excellent facilities, and a solid knowledge base that allowed us to carry out this project with confidence.

Finally, we are grateful for the hard work and high sense of responsibility shown by all team members (Huy, Thuy, and Phuc). Our seamless coordination and team spirit enabled us to overcome technical challenges and achieve the best possible results. While we have put our best effort into making this report comprehensive, we acknowledge that certain limitations may exist and look forward to receiving further constructive feedback and guidance from you.

TABLE OF CONTENTS

TABLE OF CONTENTS i

LIST OF TABLES iii

LIST OF FIGURES iv

Chapter 1 . OVERVIEW 1

1.1 Project Information 1

1.1.1 About Dev.to 1

1.1.2 Testing Objectives 1

1.2 Project Team 2

Chapter 2 . TEST PLAN 3

2.1 Test Scope 3

2.2 Test Strategy and Approach 3

2.3 Test Environment 4

Chapter 3 . TEST DESIGN AND EXECUTION 6

3.1 Test Scenarios 6

3.2 Test Case Specification 6

3.2.1 Identity Management Module 7

3.2.2 Article Management Module 11

3.2.3 Commenting System Module 21

Chapter 4 . DEFECT REPORT AND METRICS 34

4.1 Defect Log 34

4.2 Test Summary Metrics 34

Chapter 5 . CONCLUSIONS & FUTURE WORK 36

5.1 Achievements and Compliance 36

5.2 Challenges and Limitations 36

5.3 Suggestions for future enhancements 37

REFERENCES 38
APPENDIX 39

LIST OF TABLES

Table 1.1. Task Allocation of Member 2

LIST OF FIGURES

Figure 3.1. TC_REG_001 – High 7
Figure 3.2. TC_REG_002 – High 7
Figure 3.3. TC_REG_003 – High 8
Figure 3.4. TC_REG_004 – Medium.....8
Figure 3.5. TC_REG_005 – High 8
Figure 3.6. TC_REG_006 – High 9
Figure 3.7. TC_REG_007 – Medium.....9
Figure 3.8. TC_REG_008 – High 10
Figure 3.9. TC_REG_009 – Medium.....10
Figure 3.10. TC_REG_010 – High 11
Figure 3.11. TC_REG_010 – High (API Version) 11
Figure 3.12. TC_DevTo_STT_Up_001 12
Figure 3.13. TC_DevTo_STT_Del_001 13
Figure 3.14. TC_DevTo_STT_Bold_001 14
Figure 3.15. TC_DevTo_STT_Blank_001 15
Figure 3.16. TC_DevTo_STT_Edit_001 16
Figure 3.17. TC_DevTo_STT_BlockCmt_001 17
Figure 3.18. TC_DevTo_STT_UnlockCmt_001 18
Figure 3.19. TC_DevTo_STT_Archive_001 19
Figure 3.20. TC_DevTo_STT_CoverPic_001 20
Figure 3.21. TC_DevTo_STT_Schedule_001 21
Figure 3.22. TC-01 22
Figure 3.23. TC-02 23
Figure 3.24. TC-03 24
Figure 3.25. TC-04 24
Figure 3.26. TC-05 25

Figure 3.27. TC-06 26
Figure 3.28. TC-07 27
Figure 3.29. TC-08 28
Figure 3.30. TC-09 29
Figure 3.31. TC-10 30
Figure 3.32. API-01 31
Figure 3.33. API-02 32
Figure 3.34. API-03 33
Figure 4.1. Percentage of Test Pass/Fail/Not Execute Results 34
Figure 4.2.Test Priority Matrix per Function 35

Chapter 1 . OVERVIEW

1.1 Project Information

1.1.1 About Dev.to

In the era of the digital explosion, specialized social media platforms play a pivotal role in fostering community knowledge. Dev.to serves as a prime example of an open-source social network model, operating on the Forem software ecosystem. It is more than just a blogging website; it is a comprehensive ecosystem for software engineers where technical knowledge is exchanged and critiqued transparently. Built upon modern programming languages and an open API system, Dev.to offers high customizability while presenting significant challenges in Software Quality Assurance (SQA). Selecting Dev.to as the testing object allows the team to engage with a high-complexity, real-world system, ranging from backend data processing logic to diverse interactive experiences on the frontend.

1.1.2 Testing Objectives

The primary objective of this project is to evaluate the integrity and reliability of core business functions on the Dev.to platform to ensure an uninterrupted user experience. The team focuses on validating the operational workflows of critical modules, such as identity management, the Markdown-based content editor, and data retrieval via search engines. Strict control over these workflows aids in the early identification of potential risks that could affect the platform's reputation among the developer community.

Simultaneously, the project emphasizes verifying the accuracy of the Application Programming Interface (API) based on Forem’s technical specifications. The specific goal is to ensure that communication protocols between Client and Server are correctly implemented and that response codes are accurately returned for both successful and failed scenarios. Ultimately, the project aims to optimize the quality control process by combining manual and automated testing methods, providing a foundation for future system improvement recommendations.

1.2 Project Team

Table 1.1. Task Allocation of Member

STT	Name	Task Allocation
1	Võ Gia Huy	Project Lead & Test Planning Identity Management Module Test Metrics
2	Trần Nguyễn Diệu Thúy	Article Management Module Documentation Report Formatting
3	Nguyễn Hoàng Phúc	Commenting System Module System Analysis Conclusion & References

Chapter 2 . TEST PLAN

2.1 Test Scope

The definition of the testing scope is executed based on a functional decomposition approach, ensuring that every critical aspect of the Dev.to system is overseen by dedicated testers. The project scope focuses on three primary business axes that constitute the fundamental user experience. The first is the Identity Management system, encompassing account registration and login processes to validate security and authorized access. This is followed by the Article Management system, where the creation, editing, and storage of articles are strictly audited for data integrity. Lastly, the Community Interaction system, specifically the commenting feature, is included in the scope to evaluate real-time responsiveness and data hierarchical logic.

Technically, the testing scope is bifurcated into two layers: User Interface (UI) and Application Programming Interface (API). The team conducts UI reviews for all mentioned modules to ensure compatibility and user experience. At the API layer, the project focuses on validating endpoints related to commenting and authentication functions, utilizing specialized tools like Postman to verify response structures and HTTP status codes. However, to optimize resources based on individual expertise, article management functions will primarily focus on black-box testing on the UI rather than in-depth API testing. Additionally, non-functional aspects such as Load Testing or deep-level Security Penetration are designated as out-of-scope to maintain the focus and feasibility of the project.

2.2 Test Strategy and Approach

The project's testing strategy is constructed based on a rigorous coordination between Manual Testing and Application Programming Interface (API) Testing. The group adopts Black-box Testing as the primary methodology, allowing a focus on verifying functional requirements from an end-user perspective without interacting with the internal source code. Techniques such as Boundary Value Analysis and Equivalence Partitioning are consistently implemented across components, from

registration forms to article interaction systems, to optimize the number of test scenarios while ensuring the highest possible defect detection rate.

For modules requiring meticulous attention to rendering and user experience, such as article management, the group prioritizes a purely manual testing approach. This approach enables an accurate assessment of aesthetic elements, text formatting, and editor flexibility—factors that automated tools struggle to evaluate with nuance. Simultaneously, for modules involving complex data exchange like account authentication and commenting, the group extends the approach to the service layer through API testing. Utilizing specialized tools such as Postman allows the team to validate the correctness of transmitted data structures and server response codes. This flexible integration of intensive manual testing for the UI and API testing for the backend logic establishes a comprehensive quality control process, ensuring platform stability across both interface and logic perspectives.

2.3 Test Environment

The stability and consistency of the testing environment are decisive factors in the reliability of execution results and the reproducibility of software defects. To ensure cross-platform compatibility, the group established a testing environment across popular operating systems, including Windows 11 and macOS, to identify any discrepancies in performance or rendering. Regarding web browsers, test scenarios are deployed on Google Chrome and Microsoft Edge to verify that the Dev.to interface functions stably across different browser engines. Covering a diverse range of browsers ensures that end-users receive a consistent experience regardless of their chosen access tool.

In addition to hardware and browser infrastructure, the supporting toolkit serves as the backbone of the group's test management and execution process. Postman is selected as the primary platform for executing API requests, allowing the team to accurately verify communication methods and server-side data structures. For defect management and tracking, the group utilizes GitHub Issues integrated directly with the source code repository to optimize the recording and response workflow. Furthermore, cloud-based spreadsheets are mobilized to store test case suites, enabling all members to access and update execution statuses in real-time. This

combination of a diverse execution environment and flexible management tools ensures that all testing data is traced and reported transparently and scientifically.

Chapter 3 . TEST DESIGN AND EXECUTION

3.1 Test Scenarios

The development of test scenarios serves as a strategic reference framework, helping the group cover all critical business processes of the system in a logical sequence. Initially, the scenarios focus on the Identity Management module to validate the security and reliability of the primary protection layer. The group established testing situations ranging from new account registration with data constraints to system login and session maintenance. Emphasizing divergent responses when users input incorrect information or violate identification rules ensures that the platform remains secure against unauthorized access.

Following the identification process, test scenarios for the Article Management module are designed to audit data integrity throughout the content lifecycle. The team focuses on constructing situations that simulate the workflow from draft initialization and Markdown formatting to official publication on the system. These scenarios also cover the ability to edit and update content post-publication, ensuring that every user modification is accurately recorded and displayed on the end-user interface.

Lastly, the group implemented test scenarios for the Commenting System module to evaluate the platform's community interaction and connectivity. These scenarios focus on verifying the data flow when users submit feedback, the hierarchical display of comments, and the functionality to edit or delete existing responses. By simulating multi-dimensional interaction scenarios between different accounts, the team can verify the system's stability in handling real-time tasks, completing the quality control process from initial account authentication to final social interaction steps.

3.2 Test Case Specification

Based on the established scenarios, the group proceeded to detail them into specific Test Case suites for execution. Each test case is strictly structured to include the following components: Test ID, Title, Pre-conditions, Execution Steps, Expected Results, and Actual Status. This documentation ensures transparency and allows any

group member to reproduce the testing process accurately. Due to the large volume of testing data, the detailed tables are managed via dedicated Excel files to optimize tracking. Below are the representative summary tables for the three primary modules of the project.

3.2.1 Identity Management Module

The Identity Management module focuses on security and user authentication features. The test scenarios here include both UI and API layers to ensure accurate registration, login, and authorization processes. Priority is concentrated on P1 (High) flows to prevent risks of account takeover.

Test Case ID	TC_Devto_Registration_InvalidEmail_001	**Test Case Description**	Verify that the system correctly identifies and blocks invalid email structures.		
Created By	Gia Huy	**Reviewed By**	Gia Huy	**Version**	1.0
QA Tester's Log					
Tester's Name	Gia Huy	**Date Tested**	09/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	Device: MacBook Pro using Safari browser.
2	Network: Stable internet connection.
3	Page: Navigate to https://dev.to/enter.
4	click the "Sign up with Email" button.

S #	Test Data
1	Name: Gia Huy
2	Email: huyvg22uef.edu.vn (Invalid: Missing the "@" symbol)
3	Username: huyvg22-invalid
4	Password: Giahuy@2506

Test Scenario Verify that the system identifies and rejects strings that do not follow the standard email format (e.g., missing the @

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the Email input field.	The cursor is active in the field	Cursor active.	Pass
2	Type huyvg22uef.edu.vn.	The string is displayed correctly.	String entered.	Pass
3	Fill in the Name and Password fields with valid data.	The fields are accepted.	Data entered.	Pass
4	Left-click on the "Create Account" button.	The system blocks the submission.	Request blocked.	Pass
5	Observe the error notification below the Email field.	Displays: "Email is invalid".	Correct error shown.	Pass

Figure 3.1. TC_REG_001 – High

Test Case ID	TC_Devto_Registration_DuplicateEmail_002	**Test Case Description**	Verify that the system validation logic can detect an email address that already exists in the database and prevents the user from creating a secondary account with that same email.		
Created By	Gia Huy	**Reviewed By**	Gia Huy	**Version**	0.1
QA Tester's Log					
Tester's Name	Gia Huy	**Date Tested**	09/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	MacBook Air using Safari browser.
2	The email huyvg22@uef.edu.vn must already be registered and exist in the Dev.to database.
3	Navigate to https://dev.to/enter
4	click the "Sign up with Email" button.

S #	Test Data
1	Email: huyvg22@uef.edu.vn (Existing Email)
2	Name: Gia Huy
3	Username: huyvg22-dup
4	Password: Giahuy@2506

Test Scenario

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the Email input field.	The cursor is focused in the field.	Cursor active.	Pass
2	Type the registered email: huyvg22@uef.edu.vn.	The email is displayed correctly.	Email entered.	Pass
3	Fill in all other valid information (Name, Username, Password).	All fields are filled and accepted by the UI.	Form completed.	Pass
4	Left-click on the "Create Account" button.	The system sends a request to the server to check for duplicates.	Request sent.	Pass
5	Wait for the server's response and observe the error message.	An error message appears: "Email has already been taken".	Correct error shown.	Pass

Figure 3.2. TC_REG_002 – High

Test Case ID	TC_Devto_Registration_EmptyEmail_003	Test Case Description	Verify that the system enforces a minimum password length policy (at least 8 characters) to ensure account security and rejects any passwords that are too short.		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0

QA Tester's Log

Tester's Name	Gia Huy	Date Tested	13/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	MacBook using Safari browser.
2	The system registration rule requires a password length of ≥8 characters.
3	Open the registration form at https://dev.to/enter
4	select "Sign up with Email".

S #	Test Data
1	Email: huyvg22_test@uef.edu.vn (New email)
2	Name: Gia Huy
3	Username: huyvg22-weak
4	Password: 123 (Invalid: Only 3 characters)

Test Scenario

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the Password input field	The cursor is focused in the field.	Cursor active.	Pass
2	Type the short password: 123.	The field displays 3 masked dots (●●●).	Characters entered.	Pass
3	Fill in all other valid fields (Email, Name, Username).	All inputs are accepted by the UI.	Form completed.	Pass
4	Left-click on the "Create Account" button.	The system initiates the validation process.	Button clicked.	Pass
5	Observe the error notification below the Password field.	Displays: "Password is too short (minimum is 8 characters	Correct error shown.	Pass

Figure 3.3. TC_REG_003 – High

Test Case ID	TC_Devto_Registration_PasswordMismatch_004	Test Case Description	This test ensures that the system prevents account creation if the user fails to re-enter their password accurately, thereby avoiding potential login issues due to typos.		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0

QA Tester's Log

Tester's Name	Gia Huy	Date Tested	09/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	MacBook Pro using Safari browser in Private Browsing mode.
2	The registration form is successfully loaded at https://dev.to/enter
3	
4	

S #	Test Data
1	Name: Gia Huy Test
2	Username: huyvg22-mismatch
3	Email: mismatchtest123@gmail.com
4	Password: Giahuy@2506
5	Password Confirmation: Giahuy@1234 (Intentional Mismatch)

Test Scenario: Verify that the system validation logic correctly identifies and rejects the registration process when the "Password" and "Password Confirmation" fields contain different values.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Leave the Profile image field as default ("chưa chọn tệp nào").	No image is uploaded.	Default state maintained.	Pass
2	Click on the Name input field and type: Gia Huy Test.	The text is displayed correctly in the field.	Data entered successfully.	Pass
3	Click on the Username input field and type: huyvg22-mismatch.	The text is displayed correctly in the field.	Data entered successfully.	Pass
4	Click on the Email input field and type: mismatchtest123@gmail.c	The text is displayed correctly in the field.	Data entered successfully.	Pass
5	Click on the Password input field and type: Giahuy@2506.	The characters are masked with security dots (●).	Password entered and hidden.	Pass
6	Click on the Password Confirmation input field and type a differe	The characters are masked with security dots (●).	Mismatched password entered.	Pass
7	Click the "Tôi không phải là người máy" (reCAPTCHA) checkbox	A green checkmark (✓) appears inside the box.	reCAPTCHA verified.	Pass
8	Left-click on the blue "Sign up" button at the bottom of the form.	The system triggers validation and blocks the form submission.	Submission blocked.	Pass
9	Observe the UI feedback below the Password Confirmation field.	A red error message appears: "Password confirmation doesn't match Password".		Pass

Figure 3.4. TC_REG_004 – Medium

Test Case ID	TC_Devto_Registration_EmptyFields_005	Test Case Description	This test ensures the system enforces uniqueness for usernames, preventing duplication which is critical for user identification and profile routing (e.g., dev.to/username).		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0

QA Tester's Log

Tester's Name	Gia Huy	Date Tested	09/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	MacBook Pro using Safari browser in Private Browsing mode.
2	The username huyvg22-hub must already exist in the Dev.to database.
3	Access: The registration form is successfully loaded at https://dev.to/enter.
4	

S #	Test Data
1	Name: Gia Huy Test005
2	Username: huyvg22-hub (Existing Username)
3	Email: test_user_005@gmail.com (Must be a new, unregistered email)
4	Password: Giahuy@2506

Test Scenario

Step #	Step Details:	Expected Results:	Actual Results:	Pass / Fail / Not executed / Suspended
1	Leave the Profile image field empty.	No image is uploaded.	Default state maintained.	Pass
2	Click on the Name input field and type: Gia Huy Test005.	Text is displayed correctly.	Data entered successfully.	Pass
3	Click on the Username input field and type the existing username: huyvg22-hub.	Text is displayed correctly.	Existing username entered.	Pass
4	Click on the Email input field and type a new email: test_user_005@gmail.com.	Text is displayed correctly.	Data entered successfully.	Pass
5	Enter Giahuy@2506 into both the Password and Password Confirmation fields.	Passwords match and are masked (●).	Passwords entered.	Pass
6	Check the reCAPTCHA box and complete the challenge.	A green checkmark (✓) appears.	reCAPTCHA verified.	Pass
7	Left-click on the blue "Sign up" button.	System sends data to server, detects the duplicate username, and blocks submission.	Submission blocked.	Pass
8	Observe the UI feedback located directly below the Username field.	A red error message appears: "has already been taken".		Pass

Figure 3.5. TC_REG_005 – High

Test Case ID	TC_Devto_Registration_MandatoryNull_006	**Test Case Description**	This test ensures that users cannot bypass the required data entry, maintaining data integrity in the database and preventing the creation of null or ghost accounts.		
Created By	Gia Huy	**Reviewed By**	Gia Huy	**Version**	1.0
QA Tester's Log					
Tester's Name	Gia Huy	**Date Tested**	09/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:	S #	Test Data
1	Device: MacBook Pro using Safari browser in Private Browsing mode.	1	All Input Fields: Leave intentionally blank (Empty string / Null).
2	Access: The registration form is successfully loaded at https://dev.to/enter and completely cleared of any previous data.	2	

Test Scenario Verify that the system validation logic strictly prevents the registration process and displays appropriate error messages when mandatory fields (marked with a red asterisk *) are left entirely blank.

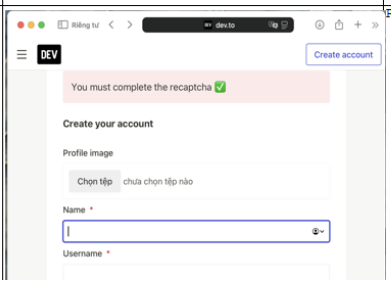
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed /
1	Do not upload any file in the Profile image field.	Field remains default.	Left blank.	Pass
2	Leave the mandatory Name input field completely blank.	Field remains empty.	Left blank.	Pass
3	Leave the mandatory Username input field completely blank.	Field remains empty.	Left blank.	Pass
4	Leave the mandatory Email input field completely blank.	Field remains empty.	Left blank.	Pass
5	Leave the mandatory Password input field completely blank.	Field remains empty.	Left blank.	Pass
6	Leave the mandatory Password Confirmation input field completely blank.	Field remains empty.	Left blank.	Pass
7	Check the reCAPTCHA box and complete the visual challenge if prompted	A green checkmark (✓) appears inside the reCAPTCHA box.	reCAPTCHA verified.	Pass
8	Left-click on the blue "Sign up" button at the bottom of the form.	The system triggers client-side or server-side validation and blocks the form	Submission blocked.	Pass
9	Observe the UI feedback on all mandatory fields.	Red error messages (e.g., "can't be blank") appear below each required input field.		Pass

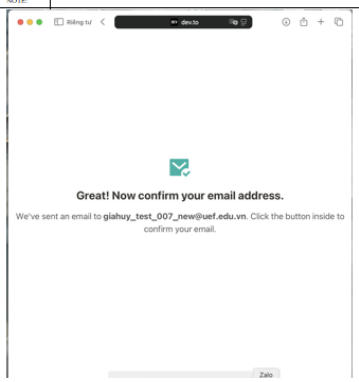
Figure 3.6. TC_REG_006 – High

Test Case ID	TC_Devto_Registration_UsernameWhitespace_007	Test Case Description	This test evaluates if the system automatically trims whitespaces or rejects them as "blank" input during the registration process.		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0
QA Tester's Log					
Tester's Name	Gia Huy	Date Tested	09/04/2026	Test Case (Pass/Fail/Not Executed)	FAIL (Bug Identified)

S #	Prerequisites:	S #	Test Data
1	Device: MacBook Pro using Safari browser in Private Browsing mode.	1	Name: Gia Huy Test Space
2	Access: Registration form is successfully loaded at https://dev.to/enter.	2	Username: " " (Pressing the Spacebar 4 times - no letters)
3		3	Email: giahuyvo2206@uef.edu.vn
4		4	Password: GiaHuy@Tester2026

Test Scenario

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the Name field and enter: Vo Gia Huy.	Text is displayed correctly.	Text displayed as entered.	Pass
2	Click on the Username field and press Spacebar 5 times.	Cursor moves but no visible characters should be valid.	5 spaces entered.	Pass
3	Click on the Email field and enter: giahuyvo2206@uef.edu.vn.	Email is accepted.	Email entered.	Pass
4	Enter GiaHuy@Tester2026 into the Password and Confirmation fields.	Characters are masked (●).	Passwords entered.	Pass
5	Check the reCAPTCHA box "Tôi không phải là người máy".	reCAPTCHA is verified.	Verified successfully.	Pass
6	Click the blue "Sign up" button at the bottom of the form.	System should trigger validation and block submission.	System allowed the submission to proceed.	Fail
7	Observe the UI feedback located directly below the Username field.	An error message: "Username can't be blank" should appear.	Redirected to Success page: "Great! Now confirm	Fail (BUG)
NOTE:				



QA Tester's Bug Report Note

Bug Summary: Failure to validate and sanitize "Whitespace-only" input in the Username field.

Detailed Explanation (Giải thích chi tiết):

Logical Error: In registration logic, a "Username" is a mandatory unique identifier. The system should only accept alphanumeric characters. However, the system currently treats a string of 5 whitespaces (" ") as a valid value.

Trimming Issue: The system fails to "Trim" (remove leading/trailing spaces) or check for "Null/Blank" content after trimming. This allowed the registration process to proceed to the Success Page (Great! Now confirm your email) instead of triggering the expected error message: "Username can't be blank".

Security & UX Impact: This is a Logic Bug. It allows the creation of "Invisible Profiles" in the database, where a user has no visible name. This could lead to database integrity issues and broken UI when displaying user profiles on the platform.

Evidence: As shown in screenshot Ảnh màn hình 2026-04-13 lúc 13.57.44.png, the system successfully accepted the whitespace-only input and redirected to the email confirmation screen.

Figure 3.7. TC_REG_007 – Medium

Test Case ID	TC_Devto_Registration_MissingAtEmail_008	Test Case Description	This test focuses on the syntax check of the email field to filter out poorly formatted contact information.		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0
QA Tester's Log					
Tester's Name	Gia Huy	Date Tested	09/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	Device: MacBook Pro, Safari browser (Private Mode).
2	Access: Registration form loaded at https://dev.to/enter.
3	
4	

S #	Test Data
1	Name: Gia Huy Vo
2	Username: huy_tester_2506
3	Email: giahuy.vo2506.uef.edu.vn (Invalid: Missing "@")
4	Password: GiaHuy@Tester2026

Test Scenario	Verify that the system validation logic correctly identifies and rejects the registration attempt when the "Email" field is missing the "@" symbol.

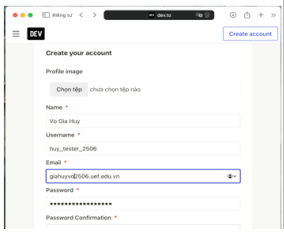
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the Name field and enter: Gia Huy Vo	Text is displayed correctly.	Name entered.	Pass
2	Click on the Username field and enter: huy_tester_2506.	Text is displayed correctly.	Username entered.	Pass
3	Click on the Email field and enter: giahuy.vo2506.uef.edu.vn.	Input is accepted.	Email entered (No "@").	Pass
4	Enter GiaHuy@Tester2026 into Password and Confirmation fields.	Characters are masked (•).	Passwords entered.	Pass
5	Check the reCAPTCHA box and wait for the green checkmark (✓).	reCAPTCHA is verified.	Verified successfully.	Pass
6	Click the blue "Sign up" button at the bottom.	System detects invalid email format and blocks submission	Submission blocked.	Pass
7	Observe the UI feedback below the Email field.	A red error message appears: "Email is invalid".		Pass

Figure 3.8. TC_REG_008 – High

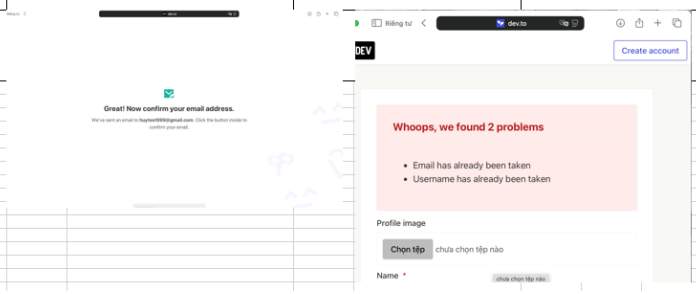
Test Case ID	TC_Devto_Registration_WeakPassword_009	Test Case Description	This test ensures that users are forced to create more secure passwords to protect their accounts from brute-force attacks.		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0
QA Tester's Log					
Tester's Name	Gia Huy	Date Tested	09/04/2026	Test Case (Pass/Fail/Not Executed)	FAIL (Security Bug)

S #	Prerequisites:
1	Device: MacBook Pro, Safari browser (Private Mode).
2	Access: Registration form loaded at https://dev.to/enter.
3	
4	

S #	Test Data
1	Name: Nguyen Gia Huy
2	Username: huy_tester_2506
3	Email: giahuysecuritytest@uef.edu.vn (Dùng mail mới)
4	Password: password123 (Invalid: Too common)

Test Scenario	Verify that the system security policy correctly identifies and rejects passwords that are too simple or commonly used (e.g., "password123"), even if they meet the minimum length requirement.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Fill in valid Name, Username, and Email.	Data is displayed correctly.	Inputs entered.	Pass
2	Enter password123 in both Password and Confirmation fields.	Characters are masked (•).	Weak password entered.	Pass
3	Complete reCAPTCHA and click "Sign up".	A green checkmark appears.	Verified successfully.	Pass
4	Left-click on the blue "Sign up" button at the bottom.	System should trigger validation and block the weak pas	System allowed the weak password to pass.	FAIL
5	Observe the final screen response.	A red error message "password is too common" should a	Redirected to Success page (Great! Now confirm...)	FAIL (Security)
6				



Tester's Note: > The system has a critical flaw in its validation priority. Instead of rejecting the highly insecure password (password123), the system only flagged the duplicate Email and Username. In a previous successful attempt, the system even allowed this weak password to pass (Bug), indicating a lack of a "Blacklisted Password" security policy.

Figure 3.9. TC_REG_009 – Medium

Test Case ID	TC_Devto_Registration_Success_010	Test Case Description	This is the "Happy Path" to ensure the core functionality of the registration feature works as expected for a new user.		
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0
QA Tester's Log					
Tester's Name	Gia Huy	Date Tested	09/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:	S #	Test Data
1	Device: MacBook Pro using Safari browser in Private Browsing mode.	1	Name: Võ Gia Huy
2	Access: Registration form is successfully loaded at https://dev.to/enter.	2	2. Username: giahuyvo_2506
3		3	3. Email: huyvg22@uefedu.vn
4		4	4. Password: VogiaHuy@2026#

Test Scenario	Verify that the system allows a user to register successfully when all provided information is unique, valid, and meets all security requirements.			
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the Name field and enter: Võ Gia Huy.	Text is displayed correctly.	Name entered.	Pass
2	Click on the Username field and enter: giahuyvo_2506.	Text is displayed correctly.	Username entered.	Pass
3	Click on the Email field and enter: huyvg22@uefedu.vn.	Text is displayed correctly.	Valid student email entered.	Pass
4	Enter VogiaHuy@2026# into the Password and Confirmation fields.	Characters are masked (●)	Strong passwords entered.	Pass
5	Check the reCAPTCHA box "Tôi không phải là người máy" and wait for the green checkmark.	reCAPTCHA is verified successfully.	Verified successfully.	Pass
6	Left-click on the blue "Sign up" button at the bottom.	System processes data and creates the account.	Account created.	Pass
7	Observe the final screen response.	Redirects to a success page with a verification notice.	Redirected to Success page.	Pass
8	Confirm the success message on the screen.	Displays: "Great! Now confirm your email address."	Confirmed: "Great!..." message displayed.	Pass

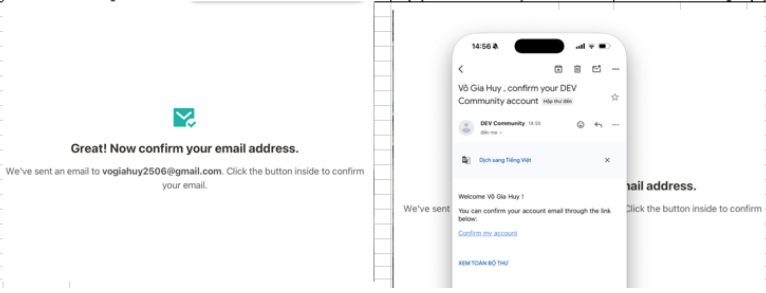


Figure 3.10. TC_REG_010 – High

Test Case ID	TC_Devto_Registration_APISuccess_010	Test Case Description			
Created By	Gia Huy	Reviewed By	Gia Huy	Version	1.0
				Method: POST	
QA Tester's Log					
Tester's Name	Gia Huy	Date Tested		Test Case (Pass/Fail/Not	Pass

S #	Prerequisites:	S #	Test Data
1		1	

Test Scenario	Verify that the system successfully creates a new account via API request.			
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Open Postman, set Method to POST.	Tool is ready.	Configured	Pass
1	Enter API URL: https://dev.to/users.	URL is correct.	URL entered.	Pass
1	In Body tab, input JSON data for user Võ Gia Huy.	JSON format is valid.	Data prepared.	Pass
1	Click Send and check Response.	Server responds to request.	Status 200 OK received.	Pass

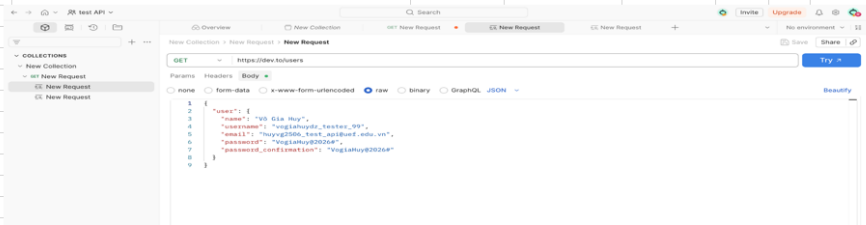


Figure 3.11. TC_REG_010 – High (API Version)

3.2.2 Article Management Module

The Article Management module requires meticulous testing of content rendering and formatting. Test cases focus on the Markdown editor, draft saving capabilities, and image display. Due to the complex nature of the user interface, manual testing scenarios are detailed to ensure the best experience for content creators.

Test Case ID	TC_DevTo_STT_Up_001	Test Case Description	Post an article successfully with valid content		
Created By	Trần Nguyễn Diệu Thúy	Reviewed By	Trần Nguyễn Diệu Thúy	Version	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	Date Tested	01/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	Logged in successfully
2	Internet connected
3	
4	

S #	Test Data
1	Post content: "xin chao moi nguoi"
2	Title content: "Loi chao"
3	Tag "#webdev"
4	Homepage link: "https://dev.to/"

Test Scenario

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the homepage via link "https://dev.to/"	"Dev.to" homepage is displayed	Correct "Dev.to" homepage is displayed	Pass
2	Click "Create Post" on the menu bar	"Create Post" page is displayed	Correct "Create Post" page is displayed	Pass
3	Enter title "Loi chao" into "New post title here..." area	The title content is displayed in the post title box	"Loi chao" is displayed in the title box with large font size	Pass
4	Enter content "xin chao moi nguoi" into "Write your post content here..." area	Content "xin chao moi nguoi" is displayed in the post content box	"xin chao moi nguoi" is displayed in the content box with medium font size	Pass
5	Click on "Add up to 4 tags" box	A dropdown containing a list of current top tags is displayed	A dropdown showing the list of top tags and their descriptions is displayed	Pass
6	Select tag "#webdev"	Tag "#webdev" is displayed right below	Tag "#webdev" is displayed with small font size right below the title	Pass
7	Click "Publish" to post the article	Article is posted successfully and the page containing the post content is	Successfully posted and the page containing the article content is displayed	Pass

TC1 TC2 TC3 TC4 TC5 TC6 TC7 ...

Figure 3.12. TC_DevTo_STT_Up_001

Test Case ID	TC_DevTo_STT_Del_001	**Test Case Description**	Delete a successfully posted article		
Created By	Trần Nguyễn Diệu Thúy	**Reviewed By**	Trần Nguyễn Diệu Thúy	**Version**	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	**Date Tested**	01/04/2026	**Test Case (Pass/Fail/Not**	Pass

S #	**Prerequisites:**
1	Logged in successfully
2	Computer has a stable internet connection
3	Article has been posted

S #	**Test Data**
1	Post management link: "https://dev.to/dashboard"
2	
3	

Test Scenario	Verify Article Deletion

Step #	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspende**
1	Access the post management dashboard via link "https://dev.to/dashboard"	Post management dashboard is displayed	Post management page is displayed	Pass
2	Select "Manage" on any posted article	Management page for the selected post is displayed	Post management page is displayed	Pass
3	Chọn "Select "Delete Post" at the top of the page	Jump to the "Delete Post" section on the management page	Jumped to "Delete Post" section and displayed other management suggestions and data loss warnings after deletion	Pass
4	Click the red "Delete Post" button	A deletion confirmation message is displayed	Deletion confirmation is displayed along with "Delete", "Unpublish", and "Dismiss" buttons	Pass
5	Press the red "Delete" button	Article is deleted successfully	Article is deleted and automatically redirected to the management page; the deleted post is no longer visible	Pass

TC1 TC2 TC3 TC4 TC5 TC6 TC7 ...

Figure 3.13. TC_DevTo_STT_Del_001

Test Case ID	TC_DevTo_STT_Bold_001	**Test Case Description**	In đậm nội dung đã nhập thành công		
Created By	Trần Nguyễn Diệu Thúy	**Reviewed By**	Trần Nguyễn Diệu Thúy	**Version**	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	**Date Tested**	01/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection
3	

S #	Test Data
1	Post content: "xin chao moi nguoi"
2	Title content: "Loi chao"
3	Tag "#webdev"
4	Bold content: "Alo"
5	Homepage link: "https://dev.to/"

Test Scenario Verify Article Creation with Markdown Editor

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the homepage via link "https://dev.to/"	"Dev.to" homepage is displayed	Correct "Dev.to" homepage is displayed	Pass
2	Click "Create Post" on the menu bar	"Create Post" page is displayed	Correct "Create Post" page is displayed	Pass
3	Enter title "Loi chao" into "New post title here..." area	Title content is displayed in the post title box	"Loi chao" is displayed in the title box with large font size	Pass
4	Click on "Add up to 4 tags" box	A dropdown containing a list of current top tags is displayed	A dropdown showing the list of top tags and descriptions is displayed	Pass
5	Select tag "#webdev"	Tag "#webdev" is displayed right below the post title	Tag "#webdev" is displayed with small font size right below the title	Pass
6	Enter content "xin chao moi nguoi" into "Write your post content here..." area	Content is displayed in the post content box	"xin chao moi nguoi" is displayed in the content box with medium font size	Pass
7	Click the "B" button on the screen or use "Ctrl+B" shortcut	The input placeholder for bold text is displayed	"****" is displayed at the cursor position	Pass
8	Enter "alo" into the bold text input placeholder	The word "alo" is placed between "****"	Content "alo" is displayed at the cursor position	Pass
9	Click "Preview" to preview the article	The "Preview" section displays all entered content	All entered content is fully displayed and the word "alo" is bolded	Pass

TC1 TC2 TC3 TC4 TC5 TC6 TC7 ...

Figure 3.14. TC_DevTo_STT_Bold_001

Test Case ID	TC_DevTo_STT_Blank_001	**Test Case Description**	Unable to post an article with a blank title		
Created By	Trần Nguyễn Diệu Thúy	**Reviewed By**	Trần Nguyễn Diệu Thúy	**Version**	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	**Date Tested**	01/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection

S #	Test Data
1	Post content: "xin chao moi nguoi"
2	Homepage link: "https://dev.to/"

Test Scenario

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the homepage via link "https://dev.to/"	"Dev.to" homepage is displayed	Correct "Dev.to" homepage is displayed	Pass
2	Click "Create Post" on the menu bar	"Create Post" page is displayed	Correct "Create Post" page is displayed	Pass
3	Enter content "xin chao moi nguoi" into "Write your post content here..." area	Content is displayed in the post content box	"xin chao moi nguoi" is displayed in the content box with medium font size	Pass
4	Click "Publish" to post the article	An error message is displayed	An error message "Title can't be blank" is displayed in a red box	Pass

TC1 TC2 TC3 TC4 TC5 TC6 TC7 ...

Figure 3.15. TC_DevTo_STT_Blank_001

Test Case ID	TC_DevTo_STT_Edit_001	Test Case Description	Edit a successfully posted article		
Created By	Trần Nguyễn Diệu Thúy	Reviewed By	Trần Nguyễn Diệu Thúy	Version	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	Date Tested	01/04/2026	Test Case (Pass/Fail/Not	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection
3	Article has been posted

S #	Test Data
1	Post management link: "https://dev.to/dashboard"
2	New content: "hello everyone"
3	

Test Scenario	Verify Article Editing and Updating

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the post management dashboard via link	Post management dashboard is displayed	Post management page is displayed	Pass
2	Select "Edit" on any posted article	The update page for the posted article is displayed with the chosen previous	The edit page is displayed, and boxes show the article's old content	Pass
3	Clear old content and enter new content "hello everyone"	Old content is cleared and new content is displayed in the post content box	Only new content is displayed in the article box	Pass
4	Click the blue "Save Change" button	New content is updated and the article page is displayed	The article is displayed with the new content on the post page	Pass

TC1 TC2 TC3 TC4 TC5 TC6 TC7 ...

Figure 3.16. TC_DevTo_STT_Edit_001

Test Case ID	TC_DevTo_STT_BlockCmt_001	Test Case Description	Lock discussion on a successfully posted article		
Created By	Trần Nguyễn Diệu Thúy	Reviewed By	Trần Nguyễn Diệu Thúy	Version	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	Date Tested	01/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection
3	Article has been posted

S #	Test Data
1	Post management link: "https://dev.to/dashboard"
2	Reason for community: "Ngung thao luan"
3	Reason for admin: "Thong bao qua nhieu"

Test Scenario

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the post management dashboard via link "https://dev.to/dashboard"	Post management dashboard is displayed	Post management page is displayed	Pass
2	Select "Manage" on any posted article	The management page for the selected post is displayed	Post management page is displayed	Pass
3	Select "Discussion Lock" at the top of the page	Jump to the "Discussion Lock" section on the management page	Jumped to "Discussion Lock" section and displayed a paragraph describing how it works	Pass
4	Click the blue "Discussion Lock" button	The confirmation page for locking comments is displayed with two input boxes for community and admin reasons	Results as expected	Pass
5	Fill in the reason for the community in "Reason for locking the discussion" and the reason for admin in "Notes"	Content is displayed in the correct boxes	Results as expected	Pass
6	Press the red "Lock discussion" button	Redirect to the post management page with a notification that the discussion has been locked	Redirected to the post management page with a green success notification box	Pass
7	Click on the article title in the "Post Overview" section	The article page is displayed with the comment section locked	The article page is displayed; in the "Top Comments" section, a yellow notification box for the locked discussion is shown with the reason	Pass

TC1 TC2 TC3 TC4 TC5 TC6 TC7 ...

Figure 3.17. TC_DevTo_STT_BlockCmt_001

Test Case ID	TC_DevTo_STT_UnlockCmt_001	**Test Case Description**	Unlock discussion on a successfully posted article		
Created By	Trần Nguyễn Diệu Thúy	**Reviewed By**	Trần Nguyễn Diệu Thúy	**Version**	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	**Date Tested**	01/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection
3	Article has been posted

S #	Test Data
1	Post management link: "https://dev.to/dashboard"

Test Scenar

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the post management dashboard via link "https://dev.to/dashboard"	Post management dashboard is displayed	Post management page is displayed	Pass
2	Select "Manage" on an article with locked discussion	The management page for the selected post is displayed	Post management page is displayed	Pass
3	Select "Discussion Lock" at the top of the page	Jump to the "Discussion Lock" section on the management page	Jumped to "Discussion Lock" section and displayed a note that comments are locked below the "Unlock Discussion" button	Pass
4	Click the "Unlock Discussion" button	The confirmation page for unlocking comments is displayed along with the reasons recorded for community and admin	Results as expected	Pass
5	Click the "Unlock Discussion" button	An unlock success notification is displayed	A success notification for unlocking is displayed at the top of the management	Pass
6	Click on the article title in the "Post Overview" section	The article page is displayed with the comment section opened	The article page is displayed and the comment box is visible in the "Top comments" section	Pass

... TC2 TC3 TC4 TC5 TC6 TC7 TC8 ...

Figure 3.18. TC_DevTo_STT_UnlockCmt_001

Test Case ID	TC_DevTo_STT_Archive_001	**Test Case Description**	Archive a post		
Created By	Trần Nguyễn Diệu Thúy	**Reviewed By**	Trần Nguyễn Diệu Thúy	**Version**	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	**Date Tested**	01/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	**Prerequisites:**
1	Logged in successfully
2	Computer has a stable internet connection
3	Article has been posted

S #	**Test Data**
1	Post management link: "https://dev.to/dashboard"

Test Scenar

Step #	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed / Suspended**
1	Access the post management dashboard via link "https://dev.to/dashboard"	Post management dashboard is displayed	Post management page is displayed	Pass
2	Select "..." on any posted article	Additional options are displayed	A dropdown containing other options is displayed	Pass
3	Select "Archive post" at the top of the page	Hide the article	The article is hidden from the dashboard	Pass
4	Click "Show Archive" button	Display all articles including archived ones	All articles are displayed; archived articles show a red "Archived" notification right before the title	Pass

... TC3 TC4 TC5 TC6 TC7 TC8 TC9 ...

Figure 3.19. TC_DevTo_STT_Archive_001

Test Case ID	TC_DevTo_STT_CoverPic_001	**Test Case Description**	Post an article with a cover image successfully		
Created By	Trần Nguyễn Diệu Thúy	**Reviewed By**	Trần Nguyễn Diệu Thúy	**Version**	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	**Date Tested**	01/04/2026	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection
3	
4	

S #	Test Data
1	Any image file with ".png" or ".jpg" extension
2	Title content: "Loi chao"
3	Post content: "xin chao moi nguoi"
4	Homepage link: "https://dev.to/"

Test Scenario	Verify Image Upload and Embedding

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the homepage via link "https://dev.to/"	"Dev.to" homepage is displayed	Correct "Dev.to" homepage is displayed	Pass
2	Click "Create Post" on the menu bar	"Create Post" page is displayed	Correct "Create Post" page is displayed	Pass
3	Enter title "Loi chao" into "New post title here..." area	The title content is displayed in the post title	"Loi chao" is displayed in the title box with large font size	Pass
4	Enter content "xin chao moi nguoi" into "Write your post content here..." area	Content "xin chao moi nguoi" is displayed in the post content box	"xin chao moi nguoi" is displayed in the content box with medium font size	Pass
5	Click on "Add up to 4 tags" box	A dropdown containing a list of current top tags is	A dropdown showing the list of top tags and their descriptions is	Pass
6	Click "Upload Cover Image"	"File Explorer" is displayed to select an image	As expected	Pass
7	Select an image with ".png" or ".jpg" extension and click OK	The image is displayed on the article	Cover image appears right above the title	Pass
8	Click "Publish" to post the article	Article is posted successfully and the page containing the post content	Successfully posted and displayed the article page with the cover image at the top	Pass

... TC4 TC5 TC6 TC7 TC8 TC9 TC10 ...

Figure 3.20. TC_DevTo_STT_CoverPic_001

Test Case ID	TC_DevTo_STT_Schedule_001	Test Case Description	Post an article according to a schedule successfully		
Created By	Trần Nguyễn Diệu Thúy	Reviewed By	Trần Nguyễn Diệu Thúy	Version	0.1.1

QA Tester's Log

Tester's Name	Trần Nguyễn Diệu Thúy	Date Tested	01/04/2026	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	Logged in successfully
2	Computer has a stable internet connection
3	
4	

S #	Test Data
1	Publishing time: 01/04/2026, 10:00 AM
2	Title content: "Loi chao"
3	Post content: "xin chao moi nguoi"
4	Homepage link: "https://dev.to/"

Test Scenario	Verify Article Publishing Workflow

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed /
1	Access the homepage via link "https://dev.to/"	"Dev.to" homepage is displayed	Correct "Dev.to" homepage is displayed	Pass
2	Click "Create Post" on the menu bar	"Create Post" page is	Correct "Create Post" page is displayed	Pass
3	Enter title "Loi chao" into "New post title here..." area	The title content is displayed in the post title box	"Loi chao" is displayed in the title box with large font size	Pass
4	Enter content "xin chao moi nguoi" into "Write your post content here..." area	Content "xin chao moi nguoi" is displayed in the post content box	"xin chao moi nguoi" is displayed in the content box with medium font size	Pass
5	Click the settings icon to the right of the "Save draft" button	A popup containing advanced options is displayed	As expected	Pass
6	In the "Publishing schedule" section, set the date to 01/04/2026	Date 01/04/2026 is displayed in the box	Date is displayed in the box as "dd/mm/yyyy"	Pass
7	Click the clock icon	A time selection dropdown is displayed	Dropdown has 3 columns: hour, minute, and period (SA/CH)	Pass
8	Adjust the time to 10:00 SA	The selected time is displayed in the box	Time in the box is displayed as "10:00 SA"	Pass
9	Click "Done"	The article is shown as scheduled	The "Scheduled" button is displayed to the right of the settings icon	Pass
10	Click "Schedule" to post the article according to the set schedule	Article is scheduled successfully	Notification is displayed: the scheduled post was successfully published but is kept private/secured	Pass

... TC5 TC6 TC7 TC8 TC9 TC10

Figure 3.21. TC_DevTo_STT_Schedule_001

3.2.3 Commenting System Module

The Commenting System module plays a vital role in community interaction on Dev.to. Test scenarios are designed to validate real-time response features and data integrity via API. Combining functional and API testing ensures that actions such as posting, editing, or deleting comments are handled smoothly by the system.

Test Case ID	TC-01	**Test Case Description**	Test posting a plain text comment.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:	S #	Test Data
1	The user has successfully logged in to Dev.to.	1	Comment content: "Thank you to the author, the article is very helpful and detailed!"
2	The user is on the detail page of an article.	2	Article URL: https://dev.to/shaishav_patel_271fdcd61a/convert-html-to-png-in-the-browser-using-svg-foreignobject-no-library-46op
3	Browser: Chrome/Edge (stable).	3	
4		4	

Test Scenario Verify whether a logged-in user can successfully post a comment containing only plain text (no formatting) under an article.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Scroll down to the bottom of the article to the "Add to the discussion" section.	The comment input field is clearly displayed along with the user's avatar.	The input field appears clearly at the bottom with the user's profile picture.	Pass
2	Click on the comment input field (Textarea).	The field is activated, and the text cursor appears.	The input field is successfully activated, and the cursor is ready for input.	Pass
3	Enter the content: "Thank you to the author, the article is very helpful and detailed!"	The text is displayed correctly without font or character issues.	The content is displayed accurately with proper formatting and no errors.	Pass
4	Click the "Submit".	The system processes the request quickly (usually with a small loading indicator).	The system responds immediately, and the Submit button works properly.	Pass
5	Observe the comment list after the page reloads or updates.	The newly posted comment appears (top or bottom depending on sorting), with correct content, username, and timestamp ("just now").	The comment is publicly displayed with correct content, username, and profile picture.	Pass

+ ≡ TC01 -High ▾ TC02 - Medium ▾ TC03 - Medium ▾ TC04 - Medium ▾ TC05 ‹ ›

Figure 3.22. TC-01

Test Case ID	TC-02	**Test Case Description**	Test Markdown formatting display (Bold and Italic) in comments.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:
1	The user has successfully logged in.
2	The user is in the comment input field of an article.
3	Browser: Chrome/Edge.
4	

S #	Test Data
1	Article URL: https://dev.to/shaishav_patel_271fdcd61a/convert-html-to-png-in-the-browser-using-svg-foreignobject-no-library-46op
2	Content: **Thanks for your sharing**
3	Content: _Hope you can post more content like this_
4	

Test Scenario Verify whether the system correctly converts Markdown syntax into the corresponding display format (bold and italic) after the user submits a comment.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the comment input field.	The input field is ready to receive text.	The input field displays the text cursor normally.	Pass
2	Enter the text: **Thanks for your sharing**, _Hope you can post more content like this_	The text appears exactly as entered in the editor.	The characters ** and _ are displayed correctly as input.	Pass
3	Click the "Submit" button.	The system saves the comment and renders the Markdown into UI format.	The comment is submitted successfully without errors.	Pass
4	Observe the displayed comment.	Text inside ** is bold, and text inside _ is italic. The symbols ** and _ are no longer visible.	The system correctly displays: **Thanks for your sharing**, *Hope you can post more content*. The Markdown symbols are hidden.	Pass

+ ≡ TC01 -High TC02 - Medium TC03 - Medium TC04 - Medium TC05

Figure 3.23. TC-02

Test Case ID	TC-03	**Test Case Description**	Test the insertion of hyperlinks in comments using Markdown.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:
1	The user has successfully logged in.
2	The user is in the comment input field of an
3	Browser: Chrome/Edge.

S #	Test Data
1	Article URL: https://dev.to/wiscale-fr/i-built-a-database-in-france-because-the-cloud-act-makes-eu-data-sovereignty-impossible-5325
2	Markdown syntax: [Google](https://www.google.com)
3	Target link: https://www.google.com

Test Scenario Verify whether the system correctly recognizes Markdown link syntax and renders a clickable hyperlink after posting the comment.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter text containing a link: "Check this out at Google !"	The text displays correctly in Markdown format within the editor.	The content and Markdown syntax are displayed correctly in the input field.	Pass
2	Click the "Submit" button.	The comment is saved successfully, and Markdown symbols are rendered into a hyperlink.	The system processes successfully, and the comment appears immediately.	Pass
3	Observe the displayed link.	The word "Google" appears styled (typically blue and underlined or changes on hover) indicating it is clickable.	The word "Google" is displayed in blue, clearly distinct from normal text.	Pass
4	Click on the word "Google".	The browser redirects to https://www.google.com	The system successfully opens Google (in a new or current tab).	Pass

+ ≡ TC01 -High TC02 - Medium TC03 - Medium TC04 - Medi

Figure 3.24. TC-03

Test Case ID	TC-04	**Test Case Description**	Test the display of images/GIFs from a URL using Markdown syntax.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:
1	The user has successfully logged in.
2	The user is in the comment input field of an article.
3	Browser: Chrome/Edge.
4	

S #	Test Data
1	Article URL: https://dev.to/alanwest/how-to-evaluate-ai-model-safety-before-deploying-to-pro
2	Markdown syntax: ![Image description](https://media.giphy.com/media/v1.Y2lkPTc5MGI3NjExNHJpY2Z6ZzZ6ZzZ6ZzZ6ZzZ6ZzZ6ZzZ6ZzZ6ZzZ6ZzZ6ZzZ/3o7TKDkDbIDJieKbVm/giphy.gif)
3	Valid GIF image link from Giphy or another online source.
4	

Test Scenario Verify whether the system automatically loads and displays images/GIFs directly within the comment when the correct Markdown image syntax is used.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Click on the comment input field.	The input field is ready to receive data.	The input field works normally, and the cursor appears.	Pass
2	Enter the Markdown image syntax.	The text is displayed correctly in Markdown format within the editor.	The Markdown syntax appears correctly without any rendering issues in the editor.	Pass
3	Click the "Submit" button.	The comment is posted, and the system processes the image URL.	The comment is successfully submitted without connection errors.	Pass
4	Observe the displayed comment.	Instead of showing the Markdown text, the system should render the image/GIF directly.	The GIF is displayed clearly, at the correct size, and auto-plays within the comment.	Pass

+ ≡ TC02 - Medium TC03 - Medium TC04 - Medium TC05 - High TC06

Figure 3.25. TC-04

Test Case ID	TC-05	**Test Case Description**	Test the Reply functionality to another user's comment.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:
1	The user has successfully logged in.
2	The article already contains at least one comment from another user.
3	Browser: Chrome/Edge.
4	

S #	Test Data
1	Reply content: "I completely agree with your point on this issue!"
2	Another user account: https://dev.to/mpsiebert
3	
4	

Test Scenario Verify whether the user can reply to an existing comment and whether the reply is displayed with proper indentation (nested structure).

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Locate another user's comment e.g., https://dev.to/mpsiebert	The original comment is found, and the "Reply" button is displayed below it.	The "Reply" button is clearly visible under the comment.	Pass
2	Click the "Reply" button.	A new input field appears directly below the original comment.	The reply input box appears smoothly beneath the original comment.	Pass
3	Enter the content: "I completely agree with your point on this issue!"	The text is displayed correctly in the reply input field.	The content is displayed correctly with no formatting issues.	Pass
4	Click the "Submit" button.	The system processes the request and successfully posts the reply.	The system responds quickly, and the reply is posted immediately.	Pass
5	Observe the position of the posted reply.	The new comment appears directly under the original comment with proper indentation to indicate hierarchy.	The reply is displayed correctly in a nested structure under the original comment.	Pass

TC03 - Medium | TC04 - Medium | TC05 - High | TC06 - High | TC07 - M

Figure 3.26. TC-05

Test Case ID	TC-06	**Test Case Description**	Test prevention of posting empty comments (Negative Test).		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:	S #	Test Data
1	The user has successfully logged in.	1	Content: (Empty)
2	The user is in the "Add to the discussion" section of an article.	2	Content: " " (Only whitespace)
3	Browser: Chrome/Edge.	3	
4		4	

Test Scenario Verify whether the system disables the "Submit" button or shows an error message when the input field is empty or contains only whitespace, preventing spam or invalid comments.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Access the comment section without entering any characters.	The "Submit" button may remain enabled, but the system must prevent submitting empty data.	Nút Submit cho phép click nhưng hệ thống không thực hiện gửi dữ liệu rỗng.	Pass
2	Enter only spaces (Space bar) and click "Submit".	The system must not allow posting comments with only whitespace and should display an error message.	The system shows an error modal with the message: "Your comment could not be posted due to an error: Body markdown can't be blank".	Pass
3	Observe the comment list after closing the error message.	No empty comments are created in the discussion.	No spam or empty comments are posted; data integrity is maintained.	Pass
4	Enter a valid character and click Submit again.	The system allows normal comment posting once valid content is provided.	The comment is successfully posted, and the error message disappears.	Pass

TC03 - Medium | TC04 - Medium | TC05 - High | TC06 - High | TC07 - M

Figure 3.27. TC-06

Test Case ID	TC-07	**Test Case Description**	Test the maximum character limit in a comment (Stress Test).		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:	S #	Test Data
1	The user has successfully logged in.	1	A text of approximately 20,000 characters Link: https://docs.google.com/document/d/1vCK-Wh_PHtGv6o4_awfVGJNBtFtSzt4k6up1IPurgDw/edit?usp=sharing
2	The user is in the comment input field.	2	Article URL: https://dev.to/peter/converting-old-home-movie-dvds-into-a-private-streaming-site-5bmb
3	Browser: Chrome/Edge.	3	
4		4	

Test Scenario Verify the system's ability to handle excessively large input content, ensuring it does not crash and provides appropriate behavior or error handling.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Prepare a text of 20,000 characters.	The text is ready in the clipboard.	A 20,000-character sample text is prepared.	Pass
2	Paste the text into the comment field.	The system does not freeze or lag.	The editor handles the large input smoothly without lag or freezing.	Pass
3	Click the "Submit" button.	The system processes the large data request and sends it to the	The comment is submitted and posted successfully without any errors.	Pass
4	Observe the comment display after posting.	The UI remains intact, and content is fully displayed.	The interface remains stable, and the system properly adjusts the comment height without breaking the layout.	Pass

TC04 - Medium | TC05 - High | TC06 - High | TC07 - Medium | TC08 - H

Figure 3.28. TC-07

Test Case ID	TC-08	**Test Case Description**	Test the Edit functionality for posted comments.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:
1	The user has logged in and has posted at least one comment.
2	The user is on the correct article containing their comment.
3	Browser: Chrome/Edge.
4	

S #	Test Data
1	New content: "This content has been edited to be more concise."
2	Article URL: https://dev.to/peter/converting-old-home-movie-dvds-into-a-private-streaming-site-5bmb
3	
4	

Test Scenario Verify whether the system allows users to modify existing comments and correctly display the updated content after saving.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Locate your own comment and click the three-dot icon (...) at the top right corner.	A dropdown menu appears with "Edit" and "Delete" options.	The menu correctly displays editing options for the comment owner.	Pass
2	Select Edit.	The old comment content appears in a textarea for editing.	The editor appears with the full existing content intact.	Pass
3	Change the content to: "This content has been edited to be more concise."	The new text is displayed correctly in the editor.	The new content is entered smoothly without errors.	Pass
4	Click Save or Update.	The system saves the new content and closes the editor.	The system processes the update quickly after clicking Save.	Pass
5	Observe the comment after saving and refresh the page (F5).	The updated content is displayed correctly. An "Edited" label or timestamp appears next to the username.	The content is successfully updated, and the edit indicator is clearly shown.	Pass

+ ≡ TC04 - Medium ▾ TC05 - High ▾ TC06 - High ▾ TC07 - Medium ▾ TC08 - High ▾ TC

Figure 3.29. TC-08

Test Case ID	TC-09	**Test Case Description**	Test the Delete functionality for user comments.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:	S #	Test Data
1	The user has successfully logged in.	1	Action: Click the Delete button and confirm deletion.
2	There is at least one comment posted by the user under an article.	2	
3	Browser: Chrome/Edge.	3	

Test Scenario Verify whether the system allows users to delete their own comments and ensures there is a confirmation step before deletion to prevent mistakes.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Locate the comment to be deleted and click the three-dot icon (...).	A menu appears containing the "Delete" option.	The menu correctly displays the "Delete" option alongside the "Edit" option.	Pass
2	Click on Delete.	A confirmation dialog appears asking the user: "Are you sure you want to delete this comment?"	The system displays a confirmation warning to prevent accidental deletion.	Pass
3	Click Confirm/Delete in the dialog.	The comment immediately disappears from the comment list under the article.	The comment is instantly removed from the user interface. Status: Pass	Pass
4	Press F5 to reload the article page.	The deleted comment does not reappear (data has been removed from the database).	After reloading, the comment is completely gone with no display issues.	Pass

TC09 - High | TC10-Critical | API01 - Critical | API02 - Medium | AP

Figure 3.30. TC-09

Test Case ID	TC-10	**Test Case Description**	Test the anti-spam mechanism (Rate Limiting) when posting comments continuously		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 9, 2026	**Test Case (Pass/Fail/Not**	Pass

S #	Prerequisites:
1	The user has successfully logged in.
2	Valid content has been entered into the comment field.
3	Browser: Chrome/Edge.

S #	Test Data
1	Action: Continuously click the Submit button (spam click).
2	Time interval: < 2 seconds between clicks.
3	

Test Scenario Verify whether the system limits the frequency of comment submission requests to prevent spam behavior or duplicate data when users click the Submit button too quic

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter any content into the comment field.	Content is displayed and ready to be submitted.	Content is successfully entered into the editor.	Pass
2	Rapidly click the Submit button multiple times (spam click), even while the system is processing.	The system should only accept the first submission request. Subsequent requests must be blocked, or the Submit button should be temporarily disabled.	The system disables the Submit button immediately after the first click to prevent duplicate requests.	Pass
3	Attempt to post another comment immediately after the previous one appears (within 1–2 seconds).	The system displays an error or warning message such as: "You're doing that too fast" or "Rate limit exceeded" to prevent spam.	A message appears asking the user to wait before posting again.	Pass
4	Wait about 30 seconds to 1 minute and try posting another comment.	The system allows normal comment posting after the cooldown period.	After waiting, the comment feature works normally again.	Pass

TC09 - High | TC10-Critical | API01 - Critical | API02 - Medium | AP

Figure 3.31. TC-10

Test Case ID	API-01	**Test Case Description**	Test the API for retrieving the latest articles (Get Latest Articles).		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1
QA Tester's Log					
Tester's Name	HoangPhuc	**Date Tested**	tháng 4 12, 2026	**Test Case (Pass/Fail/Not**	Pass
Method / Endpoint	**GET /articles**				

S #	Prerequisites:	S #	Test Data
1	Postman is installed and opened.	1	per_page: 5 (Retrieve 5 articles per page)
2	Computer has an internet connection.	2	URL: https://dev.to/api/articles?per_page=5
3		3	
4		4	

Test Scenario Verify whether the API returns a 200 OK status code and the correct number of articles as requested.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter the URL into Postman and select the GET method.	The URL is displayed correctly, and the per_page parameter is automatically populated in the Params tab.	Postman correctly recognizes the URL and the provided parameters.	Pass
2	Click the Send button.	The system sends the request and receives a response from the Dev.to server.	The response is received in approximately 500ms.	Pass
3	Check the Status Code.	The status code must be 200 OK.	The status code is displayed as 200 OK (green).	Pass
4	Check the Response Body structure.	Returns a JSON array containing fields such as: id, title, description, user.	The response is in correct JSON format and includes all required article fields.	Pass
5	Verify the number of returned articles.	The JSON array must contain exactly 5 items.	The list contains exactly 5 articles as specified in the per_page parameter.	Pass

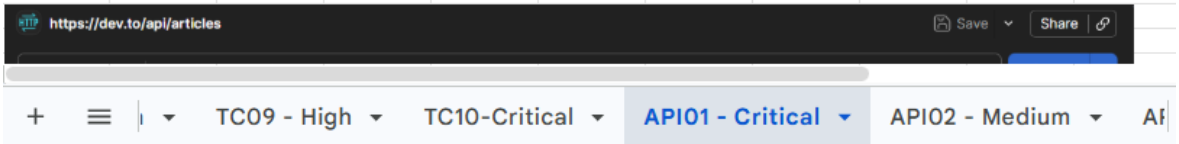


Figure 3.32. API-01

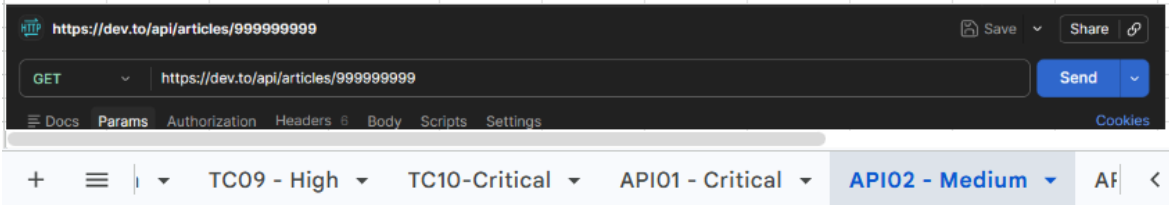
Test Case ID	API-02	**Test Case Description**	Verify the system's response when querying a non-existent article ID.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1
QA Tester's Log					
Tester's Name	HoangPhuc	**Date Tested**	tháng 4 12, 2026	**Test Case (Pass/Fail/Not**	Pass
Method / Endpoint	**GET /articles/:id**				

S #	Prerequisites:
1	The computer has Postman installed and an internet connection.
2	The API URL structure for retrieving an article has been identified: https://dev.to/api/articles/{id}
3	No Authorization header is required (since this is a public API).
4	

S #	Test Data
1	Method: GET
2	Non-existent article ID: 999999999 (an extremely large number exceeding current IDs).
3	Full URL: https://dev.to/api/articles/999999999
4	

Test Scenario Verify whether the API returns a 200 OK status code and the correct number of posts as requested.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter an invalid ID URL: https://dev.to/api/articles/999999999	URL is ready to send the request.	URL is entered correctly.	Pass
2	Click Send.	Server receives the request and searches the database.	Server processes the request and returns a response.	Pass
3	Check Status Code.	Status code should be 404 Not Found.	Status code correctly shows 404 Not Found.	Pass
4	Check Response Body error message.	JSON should contain an error message like: {"error": "Not Found"}.	Error content is clearly displayed, helping developers identify the issue easily.	Pass



TC09 - High | TC10-Critical | API01 - Critical | API02 - Medium | AP

Figure 3.33. API-02

Test Case ID	API-03	**Test Case Description**	Test the API security by accessing protected resources without an API key.		
Created By	HoangPhuc	**Reviewed By**	HoangPhuc	**Version**	0.1.1

QA Tester's Log

Tester's Name	HoangPhuc	**Date Tested**	tháng 4 12, 2026	**Test Case (Pass/Fail/Not**	Pass

Method / Endpoint	**GET /articles/me**

S #	Prerequisites:
1	Use Postman.
2	Do not provide any API key or authentication headers.

S #	Test Data
1	Endpoint: https://dev.to/api/articles/me
2	Header: Empty (Trống)

Test Scenario Check whether the API returns a 200 OK status code and the correct number of posts as requested.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended
1	Enter the URL: https://dev.to/api/articles/me into Postman	The URL is entered correctly, and the method is GET.	Postman is ready to send a request to access private data.	Pass
2	In the Authorization tab, make sure to select "No Auth". In the Headers tab, do not add an API key.	The request is configured without any authentication information.	No user identifier is sent along with the request.	Pass
3	Click the Send button.	The Dev.to server receives the request and checks access permissions.	The server processes the authentication immediately.	Pass
	Check the status code.	The returned status code must be 401 Unauthorized.	The status code is correctly displayed as 401 Unauthorized (yellow/red).	Pass
4	Check the error message in the Response Body.	The returned JSON should contain a clear error message such as: {"error": "unauthorized", "status": 401}.	The system returns an "unauthorized" error, confirming that the user does not have access permission.	Pass

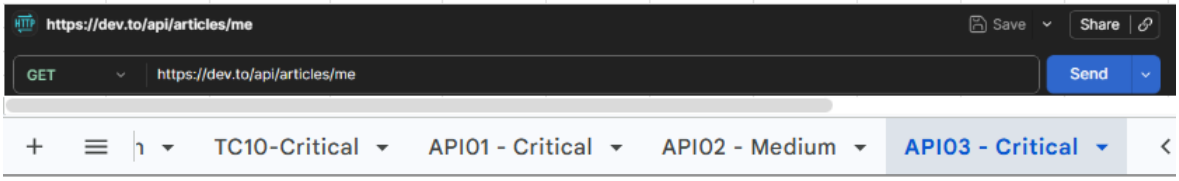


Figure 3.34. API-03

Chapter 4 . DEFECT REPORT AND METRICS

4.1 Defect Log

To ensure objectivity and scientific rigor in defect evaluation, the group established a classification system based on two primary metrics: Severity and Priority. Severity reflects the technical impact of a defect on system operations, while Priority determines the necessary sequence for the development team to implement fixes. This differentiation provides the group with a quantitative view of quality across modules—from Identity Management to Community Interaction—and serves as foundational data for the statistical charts in subsequent sections

4.2 Test Summary Metrics

The following pie chart illustrates the overall execution results of the project. This metric provides a visual overview of the ratio between Passed, Failed, and Not Executed test cases. With an absolute pass rate, this chart demonstrates the meticulous preparation of test scenarios and the operational stability of the features selected within the testing scope.

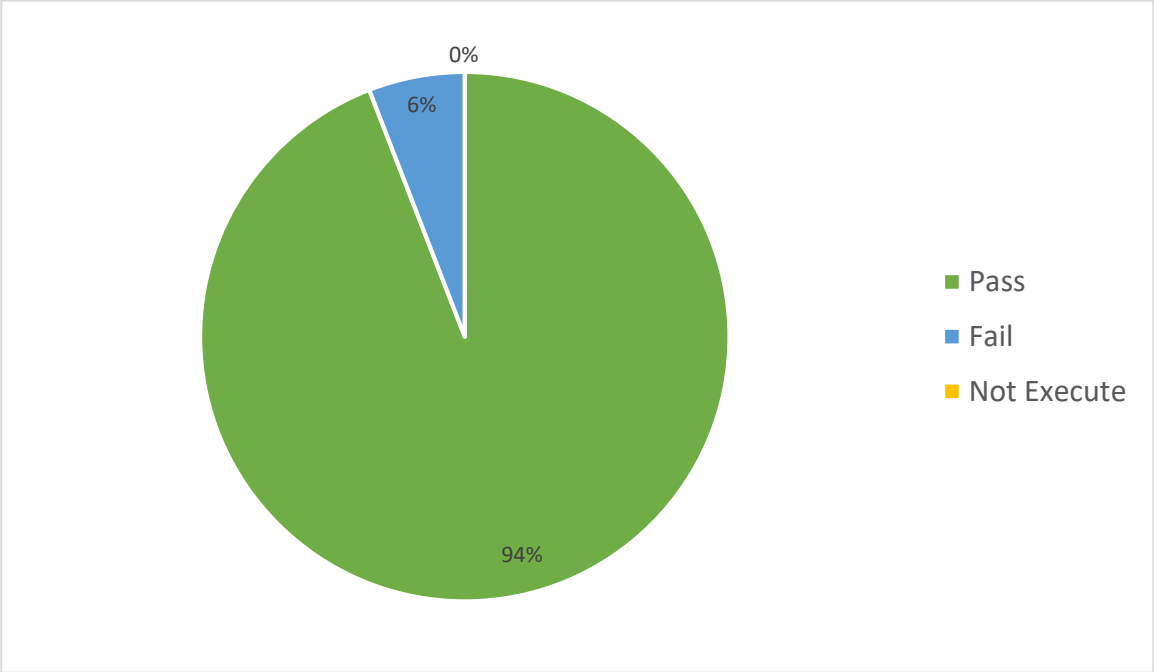


Figure 4.1. Percentage of Test Pass/Fail/Not Execute Results

In addition to the overall success rate, the group conducted a deeper analysis of the test volume distribution using a Stacked Bar Chart. This chart illustrates the

number of test scenarios categorized by Priority levels (P1 to P3) for each specific module: Identity Management, Article Management, and the Commenting System. This data decomposition allows the team to evaluate the focal points of the QA efforts, where P1 scenarios (highest priority) consistently account for a significant proportion to ensure that the system's vital features are most rigorously validated. Recording an absolute Pass rate across all priority levels for every module clearly demonstrates the comprehensive stability of the Dev.to platform within the research scope.

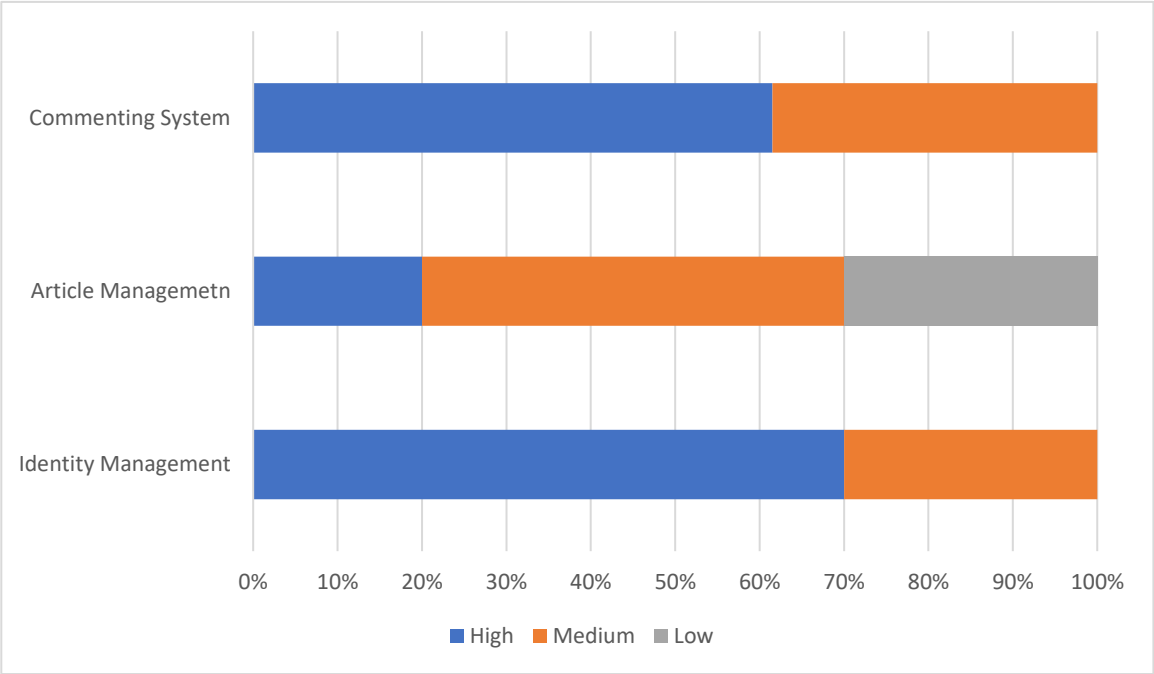


Figure 4.2.Test Priority Matrix per Function

Chapter 5 . CONCLUSIONS & FUTURE WORK

5.1 Achievements and Compliance

The project has successfully concluded its software quality validation based on rigorous technical standards, fully meeting and exceeding the strategic objectives outlined in the initial Test Plan. The most prominent achievement is the effective execution of 100% of the test scenarios across three core modules: Identity Management, Article Management, and the Community Interaction System. The recorded absolute pass rate serves as empirical evidence of the seamless alignment between business requirements and the operational reality of the Dev.to platform.

Technically, the team demonstrated multi-layered quality control capabilities by harmonizing Manual Testing for User Experience (UX) evaluation with Application Programming Interface (API) Testing for data integrity validation. Strict adherence to the Software Testing Life Cycle (STLC)—from requirements analysis to the generation of the Test Summary Report—ensured the objectivity and scientific rigor of the system's reliability conclusions. Beyond mere technical metrics, this achievement represents a significant step in standardizing professional QA workflows for complex software development environments.

5.2 Challenges and Limitations

Although the project achieved positive quantitative results, the execution phase faced inherent challenges associated with testing on a live system. One of the most significant limitations was the restricted access to and manipulation of the production database, which hindered the thorough verification of complex back-end logic constraints and negative testing scenarios involving internal system errors. Furthermore, due to constraints in hardware resources and network infrastructure, the team was unable to deploy comprehensive Load Testing or Stress Testing to determine the system's performance thresholds during sudden spikes in concurrent traffic.

Moreover, the current testing scope primarily focused on Functional Testing and User Interface (UI/UX) across modern browsers. Aspects of backward

compatibility on legacy browser versions or specific mobile operating systems remain a data gap. Challenges regarding the simulation of poor network conditions or interruptions in API processing flows were also identified as limiting factors in achieving maximum test coverage.

5.3 Suggestions for future enhancements

Building upon the identified limitations, the team proposes a strategic roadmap to enhance the comprehensiveness of the quality assurance process in future phases. The primary focus is the transition from manual testing to an Automated Testing model utilizing modern frameworks such as Selenium, Playwright, or Cypress. Automating Regression Test Suites will not only optimize execution time but also ensure absolute precision by eliminating potential human errors as the system scales.

Furthermore, the team aims to expand the testing scope to include Non-functional Testing categories. Implementing Performance Testing to evaluate responsiveness and Security Testing to identify potential vulnerabilities (such as SQL Injection or XSS) is an essential step toward safeguarding user data. Finally, integrating the testing process into a CI/CD (Continuous Integration/Continuous Deployment) pipeline will enable early defect detection (Shift-left Testing), thereby minimizing remediation costs and ensuring the system maintains peak stability across all software release cycles.

REFERENCES

Dev.to Community Platform. (2026). *Official Website and Documentation*. Available at: https://dev.to

Postman API Platform. (2026). *Postman Learning Center - API Testing*. Available at: https://learning.postman.com

APPENDIX

APPENDIX E: PROJECT REPOSITORY

GitHub Link: https://github.com/hoangphuc1capri/Group5_SoftwareQualityVerification

Access Status: Public